Project Documentation for React Native App Development

Members:

Cajes, Noreen
Marcelino, Jennie Lie
Soliman, Daniella Lyn
BSCS 3-1

Goal

Develop a mobile application using React Native by focusing on core features, intuitive UI/UX, and efficient functionality. The app idea should be chosen and customized according to your preferences and target audience. It is up to you what mobile application you will do but make it feasible.

Development Timeline

- **Start Date:** Nov 16, 2026
- **End Date:** Finals week

Project Instructions

1. Core Features

Each app idea will have its unique core features based on its purpose, but here's how to generally approach the core features:

- **Identify Key Functions:** Start by lisEng the essential features that align with the app's purpose. For example:
  - o **Habit Tracker:** Habit creation, daily habit tracking, reminders, and progress visualizaEon. o **Expense Tracker:** Adding expenses, categorizaEon, monthly limits, and summary visualizaEon.
- **Modularization:** Break down features into smaller, manageable components: o Create separate components for the user interface (like forms, buttons, and lists) and reusable components (like headers, modals, etc.).
- **User Interaction Flow:** Think about how users will interact with these features. Ensure that users can easily navigate from one feature to the next without confusion.

Prepared by:
Godwin Lorenz B. Llabres
Instructor I
DCIT 26 – APPLICATION DEVELOPMENT AND EMERGING TECHNOLOGIES

2. Intuitive UI/UX

- **Minimal and Consistent Design:**
  - Keep the design simple, focusing on essential elements on each screen.
  - Ensure colors, fonts, and spacing are consistent throughout the app.
- **User-Friendly Navigation:**
  - Implement a clear navigaEon structure using React NavigaEon for stack, tab, or drawer navigators.
  - Each screen should have a clear purpose, labeled correctly, so users know what to expect.
- **Accessibility:**
  - Ensure text is readable with appropriate contrast.
  - Consider basic accessibility features like font size adjustments, appropriate color contrast, and clear labels.

3. Efficient Functionality

- **Data Handling and Storage:**
  - Use appropriate data structures to handle data efficiently.
- **App Responsiveness:**
  - Ensure layouts are responsive across various screen sizes. UElize Flexbox, percentage based widths, and scalable fonts to make the UI adaptable.
  - Test on both Android and iOS simulators to ensure the app behaves consistently. *(Please do also note that you can choose if it is just an android or IOS app)*

- **Allowed end-product:** a must; Mobile App developed using React NaEve. Can be integrated with MERN Stack.

Prepared by:
Godwin Lorenz B. Llabres
Instructor I
DCIT 26 – APPLICATION DEVELOPMENT AND EMERGING TECHNOLOGIES

Project Documentation

Members:

Cajes, Noreen
Marcelino, Jennie Lie
Soliman, Daniella Lyn
BSCS 3-1

Table of Contents

1. Project Overview

BookReco is a book recommendation application designed to help users discover written literature, including novels, academic books, and other non-academic reading materials. The app allows users to share book recommendations by uploading titles along with ratings and feedback, making it easier for others to find books that match their interests and preferences.

The primary functionality of BookReco includes browsing and sharing recommendations, rating and reviewing books, and adding books to the favorites. The app is intended for students,

readers, and book enthusiasts who enjoy exploring new books.

By providing a user-friendly and secure platform, BookReco aims to simplify book discovery, encourage reading, and help users efficiently manage and explore meaningful literary recommendations.

2. Project Setup

2.1 Prerequisites

The following tools, software, and dependencies are required to develop the React Native application. These components support frontend development, backend services, database management, and testing on mobile devices.

Tools:

- **npm ver. 10.9.3 -** comes with Node,js and automatically downloaded along with the Node.js.
- **Expo CLI ver. 54.0.20 -** A tool to run the application in the command line. It is automatically downloaded temporarily by the npx.
- **Cloudinary for API -** It is used to manage, deploy and interact with APIs. Log in this link: Image and Video Upload, Storage, Optimization and CDN to access the tool.
- **MongoDB -** It is the NoSQL database used to store application data such as user accounts, book recommendations, ratings, and feedback. MongoDB Atlas (cloud-based) is recommended for easy setup and access.

Software:

- **Visual Studio Code (VS Code) ver. 1.107.1 -** This is the IDE used to create the application. This can be downloaded from the internet with the link of Download Visual Studio Code - Mac, Linux, Windows

- **Node.js ver. v22.20.0 -** This is used to run JavaScript and install packages. This can be downloaded from Node.js — Download Node.js®

- **Android Studio -** It is an IDE and emulator for Android. This can be downloaded from Download Android Studio & App Tools - Android Developers

- **GitHub Desktop ver.3.5.4 (x64)-** To manage and track the system code. This can be downloaded from Download GitHub Desktop | GitHub Desktop

Dependencies:

- **Frontend Dependencies:**

- **react –** Core library for building user interfaces
- **react-native –** Framework for building native mobile applications
- **expo –** Simplifies React Native development and testing
- **expo-image-picker –** Allows users to select images from their device
- **axios –** Handles HTTP requests to the backend API
- **react-navigation –** Manages navigation between screens

- **Backend Dependencies:**
  - **express** – Web framework for building REST APIs
  - **mongoose** – ODM for MongoDB database interactions
  - **cors** – Enables cross-origin resource sharing
  - **dotenv** – Manages environment variables securely
  - **jsonwebtoken** – Handles user authentication with tokens
  - **bcryptjs** – Encrypts user passwords
  - **multer** – Handles file uploads
  - **cloudinary** – Integrates Cloudinary services for media storage

2.2 Environment Setup

This section outlines the steps required to set up the development environment, starting from installing Node.js to creating and running a React Native or Expo project. It also provides guidance on configuring and using emulators or physical devices to test the application on Android platforms.

I. Install Visual Studio Code:
   1. Download Visual Studio Code either through the Microsoft Store or from the official website.
   2. Install the necessary extension for development such as JSON extension.
   3. Open the application to start coding.
   4. Use the terminal within VS Code to run commands and manage the application.

II. Install Node.js:
   1. Go to the official website of the Node Js and install the latest LTS version.
   2. Install the Node.js by following the installation instructions.
   3. Verify the installation by writing *node -v* in the terminal.

III. Initialize the NPM:
1. After installing Node.js, initialize npm by typing the following command in the terminal: *npm init -y*
2. This will automatically create a package.json file in the selected directory.
3. Verify the npm installation by typing *npm -v* in the terminal.

IV. Start REACT Native / Expo Project:
1. In the selected directory, initialize the Expo project by typing the following command: *npx create-expo-app@latest .*
2. The dot (.) indicates that the project will be installed in the current directory.

V. Start the Development Server:
1. Type the command: *npx expo start* to start the application.

VI. Run on Emulator/Physical Device:

1. **Android Emulator:**
   a. Press **a** in the terminal to run the application on the Android Emulator (Android Studio must be installed).
   b. Scan the given QR code in Expo Metro Bundler using Expo Go app
2. **Physical Device (Expo Go Application):**
   a. Install the Expo Go application from the Play Store for Android devices.
   b. Using the Expo Go application, scan the given QR code in Metro Bundler.
   c. The application will automatically open on the device, and any changes will be reflected in real time using the hot-reload feature.

3. App Structure

This section outlines the recommended project structure for the application. It defines and organizes directories and files for assets, components, screens, and navigation. Each folder should have a clear purpose to maintain code organization, improve readability, and facilitate scalability as the application grows.

I. Backend Directory

- backend/ - This directory contains all the server logic.
  - node_modules/ - This stores all the Node.js dependencies needed in the backend.
  - src/ - contains all the application logic related to the API.
    - lib/ - contains the utility and config files
      - cloudinary.js - upload image configuration.
      - cron.js
      - db.js - handles database connection.
  - middleware/- contains custom middleware functions
    - auth.middleware.js - Handles the authentication protection and token verification.
  - models/ - database models
    - Book.js - Model for book data.

Prepared by:
Godwin Lorenz B. Llabres
Instructor I

        - User.js - Model for user data.
    - routes/ - contains API routes
        - authRoutes.js - login and signup endpoints.
        - bookRoutes.js - CRUD endpoints for books.
    - ➔ index.js - server entry point
    - ➔ .env - stores environment variables.
    - ➔ .gitignore - files ignored from the version control
    - ➔ package-lock.json - dependency and script management
    - ➔ package.json- dependency and script management

II. Mobile Directory

- mobile/ - Folder for the frontend built with Expo and React Native.
    - .expo/ - contains the Expo internal files and cache.
    - app/ - Main folder for the screen and navigation layouts.
        - (auth) - authentication screen
            - **_layout.jsx -** navigation layout of screen
            - **index.jsx -** login screen
            - **signup.jsx -** registration screen
        - (tabs) - tabs for the navigation
            - **layout.jsx -** bottom tab layout
            - **create.jsx -** Home Screen
            - **favorites.jsx** - favorites page
            - **index.jsx -** create a new book .
            - **profile.jsx -** user profile.
        - ➔ **_layout.jsx**
        - ➔ **edit.jsx**
    - assets/
        - fonts/ **-** contains all the fonts used.
        - images/ **-** contains all the images used..
        - styles/ **-** contains files for styling the application on different screens
            - create.styles.js
            - home.styles.js
            - login.styles.js
            - profile.styles.js
            - signup.styles.js
    - components/ **-** contains files for reusable user interface components.
        - Loader.jsx - for loading indicator.
        - LogoutButton.jsx
        - ProfileHeader.jsx
        - SafeScreen.jsx - safe area wrapper component.
    - constants/ **-** constants used in the application
        - api.js - backend API base urls and routes
        - colors.js **-** color palette of the application

Prepared by:
Godwin Lorenz B. Llabres
Instructor I
DCIT 26 – APPLICATION DEVELOPMENT AND EMERGING TECHNOLOGIES

- lib/ - Utility functions
  - utils.js - helper logic
- node_modules/
- store/ - State management
  - authStore.js - authentication state logic
  - bookStore.js - keep track of all the books in the application
  - favoriteStore.js -keep track of books marked as favorite

➔ .gitignore
➔ app.json - Expo configuration
➔ eslint.config.js
➔ expo-env.d.ts - type definitions (Expo + TS)
➔ package-lock.json
➔ package.json
➔ README.md - project overview
➔ tsconfig.json - TypeScript compiler options

4. Core Features

This section describes the core features of the application, highlighting their main functionalities and the typical user flow for each. Each feature is broken down into smaller components or modules to show how the parts work together cohesively.

I. User Authentication (Login and Signup)

Expected Functionality:

The application should allow users to register and securely log in. Only registered users can access the main features of the app.

User Flow:

1. The user opens the app and is directed to the login screen.
2. If the user does not have an account, they navigate to the signup screen.
3. The user enters the required information for signup and submits the form.
4. The application sends the credentials to the backend API for verification.
5. Upon successful authentication, the user is redirected to the home screen.

Components:

- ➢ **LoginForm:** Input fields for email/username and password, login button, success or error handling messages.
- ➢ **SignupForm:** Input fields for name, email, and password, including a password confirmation field, along with a submit button.
- ➢ **AuthAPI:** Handles communication with the backend for login/signup requests.
- ➢ **AuthContext / State Management:** Maintains authentication status across the app.

II. Home Screen

Expected Functionality:

Displays a list of book recommendations from other users. Users can view detailed information about each book recommendation.

User Flow:

1. After login, the user is redirected to the home screen.
2. The application fetches book recommendation data from the backend.
3. The list of recommendations is displayed on the screen.
4. Users can browse and view individual books for details.

Components:

- ➢ **BookList:** Displays all book recommendations in a scrollable list.
- ➢ **BookCard / BookItem:** Individual book preview with title, author, image, rating, and short feedback.
- ➢ **BookDetails Modal / Screen:** Shows detailed information when a book is selected.
- ➢ **API Fetch Service:** Retrieves book data from the backend.

III. Book Creation

Expected Functionality:

Allows users to create and submit book recommendations, including title, author, image, ratings, and feedback.

User Flow:

1. The user clicks the ‘Create’ tab in the bottom navigation.
2. The user fills in the book title, author, rating, image, and comments.
3. After completing the form, the user clicks the ‘Submit’ button.
4. The application sends the data to the backend.
5. The new recommendation appears on the home screen.

Components:

- ➢ **BookForm:** Input fields for title, author, rating, image upload, and feedback.
- ➢ **ImageUploader:** Handles uploading book images to Cloudinary or other storage services.
- ➢ **BookAPI:** Sends new book data to the backend.
- ➢ **FormValidation:** Ensures required fields are correctly filled before submission.

IV. User Profile

Expected Functionality:
Allows users to view their profile information and log out of the application.

User Flow:

1. The user clicks the 'Profile' tab in the bottom navigation.
2. The application fetches and displays user profile information.
3. The user can log out of the app.
4. Upon logging out, the user is redirected to the login screen.

Components:

- ➢ **ProfileView:** Displays user information such as name, email, and profile picture.
- ➢ **LogoutButton:** Logs the user out and updates authentication state.
- ➢ **ProfileAPI:** Fetches and updates user data from the backend.

V. Navigation System

Expected Functionality:
Enables smooth navigation across the main features using bottom tabs.

User Flow:

1. Unauthorized users can only access the login/signup screens.
2. Authenticated users can switch between Home, Create, Profile, Add to Favorites, and Logout tabs.
3. The navigation system ensures consistent UI behavior across all screens.

Components:

- ➢ **BottomTabNavigator:** Controls the bottom tab navigation.
- ➢ **ScreenNavigator / StackNavigator:** Manages screen transitions within each tab.
- ➢ **AuthGuard / Conditional Routing:** Ensures only authorized users can access the main features.

➢ **NavigationContext:** Maintains current screen and navigation state.

5. UI/UX Design Principles

This section defines the key design principles to follow when building the BookReco application's interface. It provides guidance on maintaining visual consistency through color schemes, typography, and layout spacing, while also addressing accessibility and usability practices. Additionally, it outlines the navigation structure and interactive elements, such as loaders and animations, to ensure a smooth, intuitive, and engaging user experience across all screens.

Color Consistency: The BookReco interface uses a soft purple-based color palette to create a calm, friendly, and readable interface for user experience. Colors are applied across profile sections, buttons, cards, icons, and navigation to reinforce brand identity and improve usability.

- **Primary Colors:**
  Use Medium Slate Blue (#7C4DFF) for primary UI elements such as action buttons (e.g., Logout button), active navigation icons, and key interactive components. This color represents the main brand identity and draws user attention to important actions.

- **Secondary Colors:**
  Use Light Lavender (#F3EFFF) as the main screen background and Soft Card White (#FAF8FF) for cards and containers, such as profile information cards and recommendation items.
  For text, use Deep Purple (#4A2C82) for headings and usernames, and Muted Lavender (#8E7CC3) for secondary text like emails, dates, and metadata.

- **Accent Colors:**
  Use Soft Violet (#D1C4E9) for borders, dividers, and subtle UI outlines. Accent colors should be applied sparingly to highlight icons (edit/delete), ratings, or section separators without overpowering the primary color.

- **Background & Text Contrast:**
  The application uses light lavender and soft white backgrounds to ensure readability while maintaining a calm and visually pleasing interface. Text elements are displayed using darker purple tones to create sufficient contrast against the background.
  Dark text colors such as Deep Violet (#2E1A47) and Primary Text Purple (#4A2C82) are applied to headings, labels, and user-generated content, ensuring that text remains legible on light background such as #F3EFFF and #FAF8FF. Placeholder text uses lighter shades like #7A6FA8 to distinguish inactive fields while still remaining readable.

- **Consistency Across Screens:**
  The application maintains visual consistency across all screens, including the profile page, home feed, and book creation screen. Background colors, card styles, input fields, and button designs follow a unified design system to provide a seamless user experience.

  Light backgrounds are consistently used for screens, while cards and input containers use soft white tones with rounded corners and subtle borders. Primary action buttons, such as Logout and Share, consistently use Medium Slate Blue (#7C4DFF) to reinforce key actions. Icons, rating stars, and navigation indicators follow the same color scheme to create familiarity across interactions.

Layout and Spacing:

The application uses a consistent spacing system to ensure visual balance, clarity, and uniformity across all screens. Standardized padding, margins, and alignment improve readability and contribute to a clean, organized user interface. The layout also incorporates a SafeScreen component to ensure that content does not overlap with device notches, status bars, or system navigation areas.

Grid and Layout System:
The interface follows an 8px-based spacing system, using values such as 8px, 12px, 16px, and 24px to maintain consistency. Layouts are built using Flexbox, which is the default layout system in React Native, allowing flexible alignment, responsive positioning, and proper distribution of elements across different screen sizes.

Margins:
Vertical spacing between major sections typically ranges from 16px to 24px, providing clear separation between components such as headers, cards, and action buttons.

Padding:
Internal padding within cards, containers, and buttons usually falls between 12px and 16px, ensuring comfortable spacing between text, icons, and images.

Alignment:
Text content and headings are left-aligned for readability, while images (such as profile pictures and book covers) are vertically centered within their containers. Action icons follow consistent alignment patterns to support easy access and a cohesive visual flow.

Typography:
This is designed to ensure readability, hierarchy, and consistency throughout the application.

- **Fonts**
  - **JetBrainsMono-Medium** – Used for headings, usernames, section titles, and important

labels to provide emphasis and clarity.
    - **SpaceMono-Regular** – Used for body text, captions, metadata, and supporting content to ensure readability.
- **Sizes**
    - **Headings** (e.g., usernames, section titles): 18–20pt
    - **Subheadings** (e.g., book titles): 16pt
    - **Body text (**e.g., captions, emails): 14pt
    - **Metadata** (e.g., dates, secondary information): 12pt
- **Line Height**

  The line spacing follows proportional sizing appropriate for mobile screens, ensuring comfortable reading and preventing visual clutter.
- **Weight and Style**
    - **Medium weight**: Headings and titles (JetBrainsMono-Medium)
    - **Regular weight**: Body text and captions (SpaceMono-Regular)
    - Emphasis is applied through font weight rather than excessive styling to maintain visual consistency.

Accessibility Practices:
This feature makes the app more usable for all the users.

- **Color Contrast**:
  Text colors maintain a minimum contrast ratio of 4.5:1 against light backgrounds to ensure readability. Primary text uses darker tones, while secondary text remains readable without reducing clarity.
- **Alt Text**:
  Images such as profile photos and book covers use placeholders or icons when unavailable to maintain clarity.
- **Interactive Element Size**:
  Buttons and touchable elements meet a minimum size of 44px, improving usability on touch devices.
- **Keyboard Navigation:**
  Input fields and interactive components are structured to support keyboard navigation where applicable.
- **Error Feedback**:
  Clear text-based feedback and visual indicators are used to communicate errors or empty states.

Navigation Structure:
This ensures smooth and intuitive movement throughout the application.

- **Menu Type:**
  The app uses a **bottom tab navigation** system for primary screens.
- **Navigation Flow:**
  Signup → Login → Home → Create → Home → Favorites → Profile → Logout
- **Back/Home Buttons:**
  Navigation actions are consistently placed and easily accessible to maintain user orientation.

Prepared by:
Godwin Lorenz B. Llabres
Instructor I
DCIT 26 – APPLICATION DEVELOPMENT AND EMERGING TECHNOLOGIES

6. Efficient Functionality

Instruction: Explain best practices for achieving efficiency in code structure and performance. Include instructions on breaking down UI elements into reusable components, optimizing data handling and storage, and using appropriate tools for state management and error handling. Add guidance for responsive layouts and reducing app load times.

This section outlines the recommended practices for maintaining efficient functionality throughout the BookReco application. It focuses on organizing code into reusable and maintainable components, optimizing data handling between the frontend and backend, and ensuring smooth performance across devices.
The guidelines also cover effective state management, proper error handling, responsive layout design, and strategies to reduce loading times while delivering a consistent and user-friendly experience.

- **Reusable Components:**
  UI elements such as buttons, book cards, input fields, headers, and rating displays should be separated into reusable components. These components must follow consistent styling using the centralized color and style constants to reduce duplication and simplify maintenance as the app scales.

- **Optimized Data Handling and Storage:**
  Data fetching should be handled efficiently by requesting only necessary data from the backend. Book recommendations, user profiles, and authentication data should be stored and updated through well-defined API calls. Loading indicators should be used to avoid blank screens while data is being fetched.
- **Error Handling:**
  Errors from API requests, form submissions, and authentication processes should be handled properly. Clear feedback should be shown to users when actions fail, and the application should prevent crashes when validating inputs and handling unexpected responses.
- **Responsive Layout and Performance Optimization:**
  Flexible layouts using flex properties should be applied to ensure proper display across different screen sizes.Images and lists should be optimized to minimize load time, and unnecessary re-rendering should be avoided. Consistent spacing, shadows, and elevations help maintain consistency in performance while preserving visual clarity.
- **State Management:**
  Application state, such as user authentication status and fetched book data, should be managed consistently to ensure predictable behavior across screens. Shared states must be lifted to appropriate parent components to avoid unnecessary re-renders and improve performance.

7. Future Improvements

This section outlines potential future enhancements for the BookReco application based on user

feedback, usability observations, and performance evaluation. These improvements aim to expand functionality, enhance user experience, and improve content organization as the application grows. Suggested features focus on making book discovery more efficient, increasing personalization, and providing users with greater control over their profiles and recommendations.

- **Search bar:**
  A search feature can be added to allow users to quickly find books by title, author, or keywords. This will improve navigation efficiency, especially as the number of book recommendations increases.
- **Sorting Options:**
  Sorting functionality can be implemented to let users organize book recommendations by rating, date added, or popularity. This enhancement will help users discover content based on their preferences.
- **Profile Editing:**
  Future updates may allow users to edit their profile information, such as username or profile picture. This feature will enhance personalization and user engagement within the application.
- **Book Categorization:**
  Books can be categorized by genre, tags, or reading status to improve content organization. Categorization will make browsing easier and help users discover books that match their interests more effectively.

8. References

Burak, B. (2023). *React Native full stack app tutorial using Expo, MongoDB, and Cloudinary* [Video]. YouTube.
https://www.youtube.com/watch?v=o3IqOrXtxm8

Expo. (n.d.). *Expo documentation*.
https://docs.expo.dev

Meta Platforms, Inc. (n.d.). *React Native documentation*.
https://reactnative.dev/docs/getting-started

Node.js Foundation. (n.d.). *Node.js documentation*.
https://nodejs.org/en/docs

MongoDB, Inc. (n.d.). *MongoDB documentation*.
https://www.mongodb.com/docs

Cloudinary Ltd. (n.d.). *Cloudinary documentation*.
https://cloudinary.com/documentation

Prepared by:
Godwin Lorenz B. Llabres
Instructor I
DCIT 26 – APPLICATION DEVELOPMENT AND EMERGING TECHNOLOGIES

Axios. (n.d.). *Axios documentation*.
https://axios-http.com/docs/intro

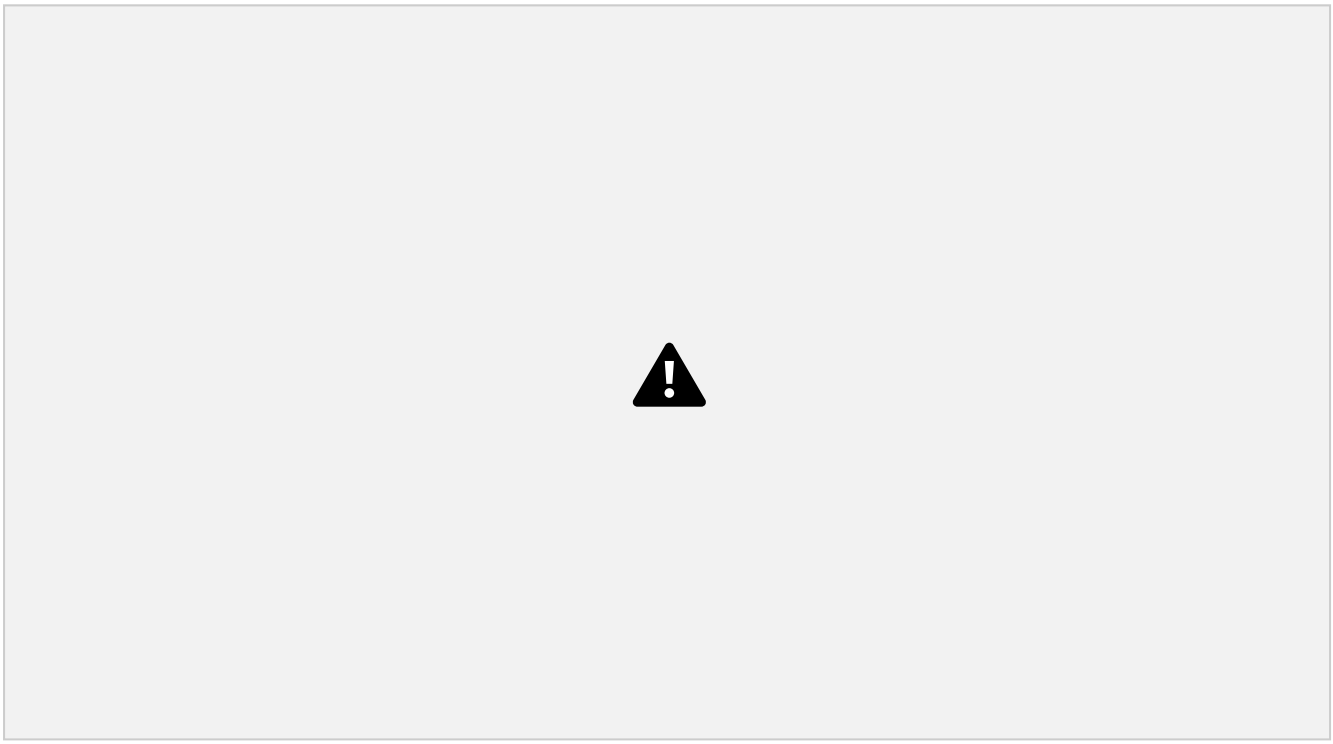
React Navigation. (n.d.). *React Navigation documentation*.
https://reactnavigation.org/docs/getting-started

Google. (n.d.). *Android Studio documentation*.
https://developer.android.com/studio

Microsoft. (n.d.). *Visual Studio Code documentation*.
https://code.visualstudio.com/docs

Grading Criteria

Prepared by:
Godwin Lorenz B. Llabres
Instructor I
DCIT 26 – APPLICATION DEVELOPMENT AND EMERGING TECHNOLOGIES



Prepared by:
Godwin Lorenz B. Llabres
Instructor I
DCIT 26 – APPLICATION DEVELOPMENT AND EMERGING TECHNOLOGIES